

Learning Expert Collaboration Topology in Mixture-of-Experts Language Models

Ben Maor
Tel Aviv University
maor1@mail.tau.ac.il

April 14, 2026

Abstract

Sparse Mixture-of-Experts (MoE) models combine the outputs of k independently acting expert networks per token, with no mechanism for experts to communicate after being selected. We propose **CT-MoE** (Collaborative Topology MoE), which augments each MoE layer with a single learned static adjacency matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ that governs message passing between the selected experts. After the k selected experts have each independently processed a token, $\mathbf{S}$ allows each expert’s output to benefit from the others it was not directly routed to, by passing their information through the experts it was. $\mathbf{S}$ is a direct `nn.Parameter` optimised end-to-end from the language modelling objective, requires no auxiliary loss, and adds only $N^2 = 256$ scalars per layer—less than 0.003% parameter overhead on a 65 M baseline.

On a mixed code and scientific-text benchmark, CT-MoE reduces perplexity by **3.6 points absolute** ($22.3 \rightarrow 18.7$; 16.1% relative) over a matched vanilla MoE baseline. A four-condition ablation cleanly isolates the mechanism: virtually the entire improvement comes from the message-passing step governed by $\mathbf{S}$, while using $\mathbf{S}$ as a routing bias alone produces a marginal degradation relative to the baseline. Analysis of the learned topology reveals emergent hub experts, a low-rank collaboration structure, and domain-specific routing specialisation. We also document five failed dynamic topology designs and trace their shared failure mode to geometric saturation in high-dimensional representation spaces, motivating the static parameter approach.

1 Introduction

Sparse Mixture-of-Experts models [5, 11] scale model capacity by routing each token to a small subset of k out of N expert networks. The fundamental operation is a *selection and combination*: choose the k experts most relevant to this token, run each independently, and form a weighted sum of their outputs.

The independence assumption built into this combination is a simplification. In a well-trained MoE, experts develop complementary specialisations [16]—one may capture syntactic structure while another processes semantic content—yet the model has no mechanism for these experts to condition on each other’s outputs for the same token. Gating weights determine *which* experts process each token; they say nothing about *how* the experts should relate to one another.

We ask: *can a learned communication topology between experts improve language modelling performance, and if so, through which mechanism?*

Our answer is yes, and the mechanism is message passing rather than routing. We introduce a single learned matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ per MoE layer, the *collaboration topology*. After the k selected experts have each independently processed the token, $\mathbf{S}$ governs a weighted message-passing

step: each expert’s output is updated with a $\mathbf{S}$ -weighted average of the other selected experts’ outputs. The key intuition is that routing determines which experts process a token but cannot change what they produce; message passing allows a token to benefit from experts it was not directly routed to by passing their outputs through the experts it was. $\mathbf{S}$ is symmetrised, diagonal-masked, and row-normalised to a valid probability distribution. It is a *static* parameter shared across all tokens and positions, optimised end-to-end from the language modelling loss.

The design is motivated by the failure of several more complex alternatives. Across five earlier designs—similarity-based dynamic graphs, output-divergence graphs, cached routing graphs, learned gates, and entropy-based auxiliary losses—we consistently observed *geometric saturation*: expert representations in high dimensions collapse to near-uniform geometry, preventing pairwise similarity matrices from learning useful structure. Static $\mathbf{S}$ sidesteps this entirely by receiving gradients directly from the task loss with no intermediate representation that can saturate.

Contributions.

1. We show that a 256-scalar learned static collaboration topology reduces MoE perplexity by 3.6 points absolute (16.1% relative) on a mixed code/text benchmark with negligible parameter overhead.
2. A four-condition ablation establishes that the gain comes almost entirely from message passing, not routing bias—a finding with direct implications for how future architectural work should allocate its budget between selection and communication mechanisms.
3. We provide topology interpretability evidence: hub experts, low-rank organisation, and domain specialisation emerge without explicit supervision.
4. We document five failed dynamic topology designs with a unified mechanistic explanation (high-dimensional geometric saturation), providing a negative result record that should guide future work on learned expert communication.

2 Related Work

Sparse Mixture-of-Experts. The modern MoE formulation originates with Shazeer et al. [11], who introduced top- k routing with noisy gating. Switch Transformer [5] simplified to $k=1$ and introduced load-balancing auxiliary losses. Subsequent work has addressed routing strategies [3, 7], capacity factors [15], and expert specialisation [1, 16]. None of these modifies how the k selected expert outputs are combined once selected.

Beyond independent expert combination. Soft MoE [9] allows all experts to contribute to each token via a differentiable dispatch matrix, but still combines expert outputs independently via their slot assignments. Mixture of Depths [10] allocates different computational depths to tokens but does not consider expert-to-expert communication. To our knowledge, CT-MoE is the first to introduce a learned inter-expert communication graph within a sparse top- k MoE layer.

Graph neural networks. The weighted message-passing step in CT-MoE can be viewed as a single-step Graph Attention Network [14] applied over the k -expert subgraph for each token, with a learned global adjacency rather than an input-dependent attention mechanism. Unlike standard GNN settings, our graph nodes are functional modules rather than explicit entities, and the topology is supervised only by task loss.

Hebbian learning. The original motivation for this work was Hebbian plasticity [6] as a learning signal for inter-expert connectivity. We ultimately abandoned this direction (Section 4) because local Hebbian rules lack the task-level error signal needed to distinguish beneficial from incidental co-activation: frequently co-activated expert pairs would strengthen their connection, biasing routing toward them still more, producing topology collapse rather than useful structure. The static **S** approach retains the conceptual intuition of stable expert relationships while using backpropagation as the learning mechanism.

Graph Mixture of Experts. Concurrent with our work, Bai et al. propose GMoE, which introduces a graph neural network router for collaborative routing in LoRA-based MoE fine-tuning [2]. Unlike CT-MoE, GMoE uses an input-dependent graph to influence expert selection, whereas our ablation shows routing bias from a learned topology contributes negligibly to performance — the gain in CT-MoE comes from post-selection message passing. GMoE also operates in the fine-tuning regime while CT-MoE targets pre-training.

3 Method

3.1 Standard MoE Layer

A standard MoE layer with N experts and top- k routing processes input $\mathbf{x} \in \mathbb{R}^{B \times T \times D}$ as follows. A router $\mathbf{R} \in \mathbb{R}^{D \times N}$ computes routing weights $\mathbf{w} = \text{softmax}(\mathbf{x}\mathbf{R})$, and the top- k expert indices $\mathcal{I} \subset [N]$ are selected per token. Each expert e applies a two-layer MLP:

$$\mathbf{o}_e = \mathbf{W}_2^{(e)} \text{GELU}(\mathbf{W}_1^{(e)} \mathbf{x}), \quad \mathbf{W}_1^{(e)}, \mathbf{W}_2^{(e)} \in \mathbb{R}^{D \times D}. \quad (1)$$

The layer output is the routing-weighted sum:

$$\text{MoE}(\mathbf{x}) = \sum_{e \in \mathcal{I}} w_e \mathbf{o}_e. \quad (2)$$

3.2 Learned Collaboration Topology

CT-MoE introduces a single parameter $\mathbf{S}_{\text{raw}} \in \mathbb{R}^{N \times N}$ per MoE layer, initialised to zero. The *normalised topology* $\mathbf{S}$ is derived through three steps applied at every forward pass:

1. **Symmetrisation:** $\mathbf{S} \leftarrow (\mathbf{S}_{\text{raw}} + \mathbf{S}_{\text{raw}}^\top)/2$. Expert i attending to j implies j attending equally to i , encoding the intuition that useful collaboration is mutual.
2. **Diagonal masking:** $\mathbf{S}_{ii} \leftarrow -\infty$. Prevents self-loops; each expert attends only to others. This is necessary because self-similarity dominates by Cauchy–Schwarz and would otherwise monopolise each row.
3. **Softmax normalisation:** $\mathbf{S} \leftarrow \text{softmax}(\mathbf{S}/\tau)$, row-stochastic. Each row defines a probability distribution over collaboration partners.

The zero initialisation makes $\mathbf{S}_{ij}^{(0)} = 1/(N-1)$ for $i \neq j$ —a uniform flat prior that places no prior assumption on which expert pairs should collaborate. All structure is learned purely from the language modelling gradient.

3.3 Message Passing

After expert execution, CT-MoE applies one step of message passing among the k selected experts. Let $\mathbf{O} \in \mathbb{R}^{B \times T \times k \times D}$ be the stacked outputs of the selected experts, and let $\mathcal{I} \in \mathbb{Z}^{B \times T \times k}$ be their indices. The $k \times k$ subgraph is extracted and re-normalised:

$$\mathbf{S}_{bti,j}^{\text{sub}} = \mathbf{S}_{\mathcal{I}_{bti}, \mathcal{I}_{btj}}, \quad \hat{\mathbf{S}}^{\text{sub}} = \mathbf{S}^{\text{sub}} \oslash (\mathbf{S}^{\text{sub}} \mathbf{1} + \varepsilon). \quad (3)$$

Messages and the residual update are:

$$\mathbf{M} = \hat{\mathbf{S}}^{\text{sub}} \mathbf{O}, \quad \mathbf{O}' = \mathbf{O} + \lambda_c \mathbf{M}, \quad (4)$$

where λ_c is the collaboration scale hyperparameter. The residual formulation ensures the baseline (no communication) is recoverable if $\mathbf{S}$ converges to uniform, making CT-MoE a strict generalisation of the standard MoE layer. The updated $\mathbf{O}'$ replaces $\mathbf{O}$ in the routing-weighted sum.

3.4 Routing Bias

$\mathbf{S}$ can also bias the router via column sums. The column sum $\mathbf{c} = \mathbf{S}^\top \mathbf{1} \in \mathbb{R}^N$ measures how much total incoming collaboration weight expert j receives from all others—a proxy for how “central” expert j is in the learned topology. Importantly, row sums of a row-stochastic $\mathbf{S}$ are identically 1.0 for all rows and would add a constant to all routing logits with no discriminative effect; column sums vary as $\mathbf{S}$ learns and carry meaningful signal:

$$\tilde{\mathbf{w}} = \text{softmax}(\mathbf{x}\mathbf{R} + \lambda_r \mathbf{c}^\top). \quad (5)$$

3.5 Ablation Variants

Four conditions isolate the contributions of each component:

Variant	Routing bias	Message passing
StandardMoE	×	×
CT-MoE-NoCollab	✓	×
CT-MoE-NoRouting	×	✓
CT-MoE-Full	✓	✓

All variants use identical Switch-Transformer load balancing [5] with coefficient $\alpha = 0.01$. $\mathbf{S}$ adds $N^2=256$ parameters per layer, totalling 1,536 across 6 layers on a 65 M model—0.0024% overhead. All four variants share the same random seed and data order, ensuring differences in perplexity are attributable to the architectural components under study rather than training stochasticity.

4 Why Static Topology? A Record of Failed Alternatives

The static $\mathbf{S}$ design emerged after systematic exploration of dynamic, input-dependent topologies. We document these failures because their shared mechanism is informative and because the negative result strengthens the case for the final design.

Attempt 1: Similarity-based dynamic graph. We computed $\mathbf{S}$ from cosine similarities between learned per-expert input projections: $S_{ij} = \tanh(\mathbf{h}_i^\top \mathbf{h}_j / \tau)$, where $\mathbf{h}_i$ are linear projections of a sequence summary vector. The projections collapsed to near-identical directions (*geometric concentration of measure* in high dimensions), freezing $|\mathbf{S}| \approx 0.59$ from early training onward. The task loss had no gradient path to the encoder through the degenerate $\mathbf{S}$.

Attempt 2: Output-divergence graph. Replacing the encoder with $S_{ij} = 1 - \cos(\mathbf{o}_i, \mathbf{o}_j)$ measured functional divergence between expert outputs. Expert outputs in $D=512$ dimensions similarly converge to near-orthogonal geometry as load balancing forces specialisation, pinning $|\mathbf{S}| \approx 0.93$ through the ceiling of the divergence measure. Gradients confirmed the metric was non-informative.

Attempt 3: Learned scalar gates. Adding scalar gates $\sigma(\gamma_r), \sigma(\gamma_c)$ initialised at $\sigma(-2) \approx 0.12$ was intended to allow the model to learn how much to trust graph signal. This created a deadlock: low gates produced weak encoder gradients; a useless graph meant gates never opened. GraphMag remained frozen at its random-initialisation value for the entire 10-epoch run.

Attempt 4: Reward-modulated Hebbian learning. A three-factor Hebbian rule updated S_{ij} proportionally to expert co-activation weighted by a task-reward signal. This increased routing concentration—frequently co-activated pairs strengthened their connection and were routed together more often in a self-reinforcing cycle—without improving perplexity, confirming the topology-collapse concern raised in Section 2.

Attempt 5: Row-entropy auxiliary losses. The auxiliary objective failed to produce useful topology structure; we were unable to determine post-hoc whether this resulted from one-hot collapse, a sign mismatch, or insufficient loss weighting, and we exclude this design from further analysis.

Shared root cause. Attempts 1–4 share a common mechanistic failure: each builds $\mathbf{S}$ from high-dimensional representations that converge to near-uniform pairwise geometry regardless of initialisation or loss design, cutting off the gradient path from the task loss to $\mathbf{S}$. Attempt 5 failed for a different and unresolved reason, but shares the same practical consequence: the auxiliary objective delivered no useful gradient signal to $\mathbf{S}$. Static $\mathbf{S}$ eliminates the failure mode of Attempts 1–4 entirely: $\mathbf{S}_{\text{raw}}$ receives gradients directly from the task loss with no intermediate representation that can saturate. The $100\times$ learning rate boost described in Section 5 is necessary to give $\mathbf{S}_{\text{raw}}$ sufficient gradient signal relative to the 65 M remaining parameters, but once this is in place, the parameter learns reliably.

5 Experiments

5.1 Setup

Architecture. A 6-layer decoder-only transformer [13] with $D=512$, 8 attention heads, $N=16$ experts, top- $k=2$, and vocabulary size 16,000. Tied token/output embeddings. Total parameters: 65.01 M. CT-MoE variants add 1,536 parameters.

CT-MoE hyperparameters. $\lambda_r = 1.5$ (routing scale), $\lambda_c = 1.0$ (collaboration scale), $\tau = 1.0$ (softmax temperature). $\mathbf{S}_{\text{raw}}$ uses a separate AdamW parameter group at learning rate 2×10^{-2} ($100\times$ the base rate of 2×10^{-4}) with no weight decay applied to $\mathbf{S}_{\text{raw}}$. The $100\times$ boost is necessary: without it, the gradient of the 256-scalar $\mathbf{S}_{\text{raw}}$ is dominated by the 65 M-parameter model and $\mathbf{S}$ learns no structure within 10 epochs. This separation of learning rates is implemented via parameter groups in AdamW and adds no architectural complexity.

Training. AdamW [8] with cosine learning rate schedule, 600-step linear warmup, minimum LR = 5% of peak. Batch size 32, sequence length 256, 10 epochs. Mixed-precision training (`torch.amp.autocast`).

Data. A 50/50 blend of Python source code (CODEPARROT-CLEAN; 12) and scientific paper text (ARXIV-SUMMARIZATION; 4), totalling 50 M characters. A SentencePiece BPE tokeniser with vocabulary size 16,000 is trained on the training split. Held-out validation sets are evaluated separately per domain. Train/validation split: 90/10 per domain.

Table 1: Validation perplexity after 10 epochs. Lower is better. **Bold** = best per column. Δ = absolute improvement over StandardMoE on the mixed set. All CT-MoE variants are parameter-matched to StandardMoE up to 1,536 scalars (0.0024% overhead).

Model	Code PPL↓	Arxiv PPL↓	Mixed PPL↓	Δ Mixed
StandardMoE	25.5	19.0	22.3	—
CT-MoE-NoCollab	26.4	18.9	22.7	−0.4
CT-MoE-NoRouting	20.0	17.7	19.0	+3.3
CT-MoE-Full	19.7	17.5	18.7	+3.6

5.2 Evaluation

Token-level perplexity is computed from the cross-entropy loss only, excluding auxiliary losses, ensuring comparability across variants. We report perplexity on three held-out sets: code-only, arxiv-only, and a 50/50 mixed split. Domain evaluation is run at end of training on the saved checkpoint.

6 Results

6.1 Main Results

Table 1 presents the main results. CT-MoE-Full achieves 18.7 mixed-domain perplexity versus 22.3 for the matched baseline—a 3.6-point absolute improvement from 1,536 additional scalar parameters. To contextualise the magnitude: this is a larger absolute gain than the difference between many architectural innovations studied at comparable parameter scales, achieved with parameter overhead below measurement precision on most model size comparisons.

Message passing is the primary driver. The ablation reveals a striking asymmetry. CT-MoE-NoCollab (routing bias only) achieves 22.7—marginally *worse* than StandardMoE (22.3), suggesting that routing bias from a partially-learned $\mathbf{S}$ slightly misguides token distribution before the topology converges. CT-MoE-NoRouting (message passing only) achieves 19.0, recovering 3.3 of the 3.6-point total gain. *Routing bias from the learned topology contributes negligibly to performance; message passing accounts for nearly the full improvement.*

This asymmetry has a natural explanation. The router already does a reasonable job selecting relevant experts; the marginal value of further biasing it toward well-connected graph nodes is low. Message passing provides qualitatively different information: it allows a token to benefit from experts it was not directly routed to by routing their outputs through the experts it was. This suggests that future MoE architectural work should invest in *post-selection communication mechanisms* rather than more elaborate routing functions.

Domain asymmetry. The gain is substantially larger on code (5.8 points) than on arxiv (1.5 points). Code has a highly compositional structure—syntax, types, semantics, and control flow are naturally separable concerns—creating stronger incentives for expert specialisation and, consequently, more benefit from inter-expert communication.

6.2 Topology Analysis

Eigenspectrum. Figure 1 shows the normalised eigenspectrum of $\mathbf{S}$ at the final layer. Strikingly, all three CT-MoE variants develop nearly identical spectra: a dominant first eigenvalue of

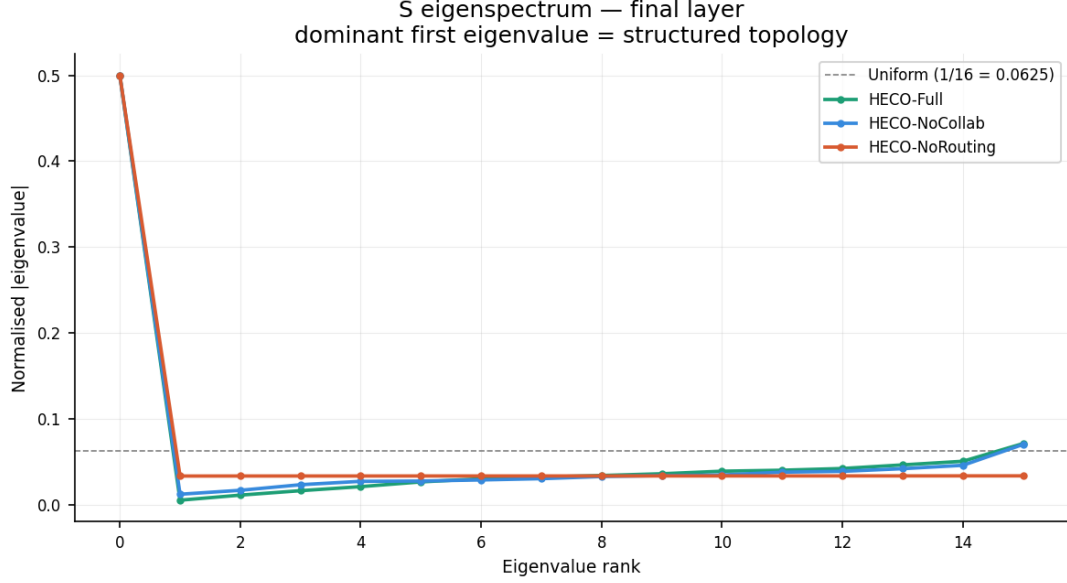


Figure 1: Normalised eigenspectrum of the learned $\mathbf{S}$ at the final layer. All three CT-MoE variants develop nearly identical near-rank-1 spectra, with a dominant first eigenvalue of approximately 0.50—eight times the uniform baseline of $1/16 = 0.0625$ —and all remaining eigenvalues near zero. The similar spectra across variants indicate that low-rank structure emerges from the symmetrisation constraint and the hub column pattern common to all three, rather than from the specific gradient signal of routing or message passing. Qualitative differences between variants are visible in the heatmaps (Figure 7) rather than the eigenspectrum.

approximately 0.50—eight times the uniform baseline of $1/16 = 0.0625$ —with all remaining eigenvalues collapsing to near zero. This near-rank-1 structure is consistent across CT-MoE-NoCollab, CT-MoE-NoRouting, and CT-MoE-Full despite their different uses of $\mathbf{S}$.

The implication is that the low-rank topology emerges from the symmetrisation constraint rather than the specific gradient signal from routing or message passing: a symmetric matrix with a dominant hub column pattern necessarily produces a near-rank-1 spectrum. The eigenspectrum therefore tells us that structure exists but not which structure—the heatmap comparison (Figure 7 and Figure 2) reveals that the three variants learn qualitatively different hub configurations despite similar spectra.

Hub experts. Figure 2 shows the learned $\mathbf{S}$ across layers for CT-MoE-Full. Layer 0 shows near-uniform structure with one isolated high-weight cell (expert $6 \rightarrow 5$, ≈ 0.40) that is an initialisation artifact, dispersing by Layer 3 as the topology reorganises. By layers 3 and 5, a clear column pattern emerges: experts 10–11 and 14–15 receive disproportionately elevated incoming collaboration weight from nearly all other experts. Concretely, the most central hub expert receives a mean column weight of approximately 0.12—nearly twice the uniform baseline of $1/15 \approx 0.067$ —with the total incoming weight concentrated on a small set of columns while most row-sender weights remain near baseline.

The variant comparison (Figure 7) reveals that different uses of $\mathbf{S}$ produce different hub configurations. CT-MoE-NoCollab, where $\mathbf{S}$ biases routing only, develops strong hubs at columns 0, 7, and 15. CT-MoE-NoRouting, where $\mathbf{S}$ governs message passing only, shows the most diffuse structure—barely above the uniform contour across most cells. CT-MoE-Full concentrates on columns 10–11 and 14–15. The fact that NoRouting develops the weakest column structure while still achieving competitive code perplexity (20.0 vs. CT-MoE-Full’s 19.7) suggests that diffuse

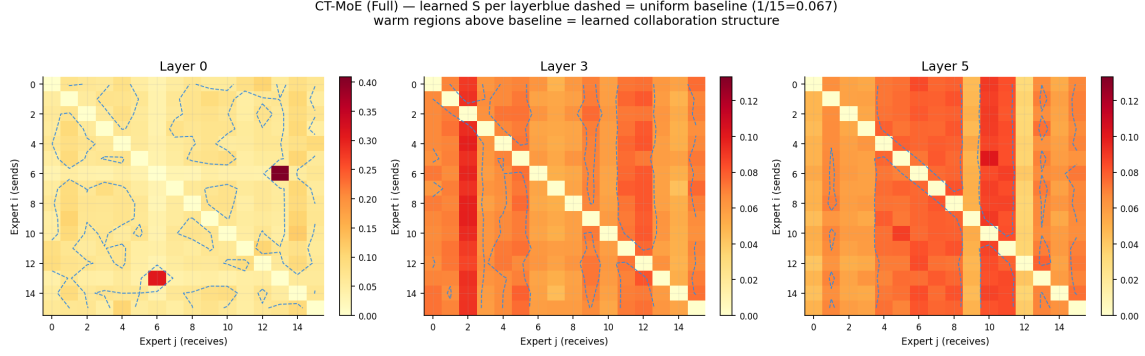


Figure 2: Learned $\mathbf{S}$ for CT-MoE-Full at layers 0, 3, and 5. Blue dashed contour marks the uniform baseline ($1/15 \approx 0.067$). By layers 3 and 5, a small set of experts receive elevated incoming collaboration weight from nearly all others, emerging as *communication hubs*.

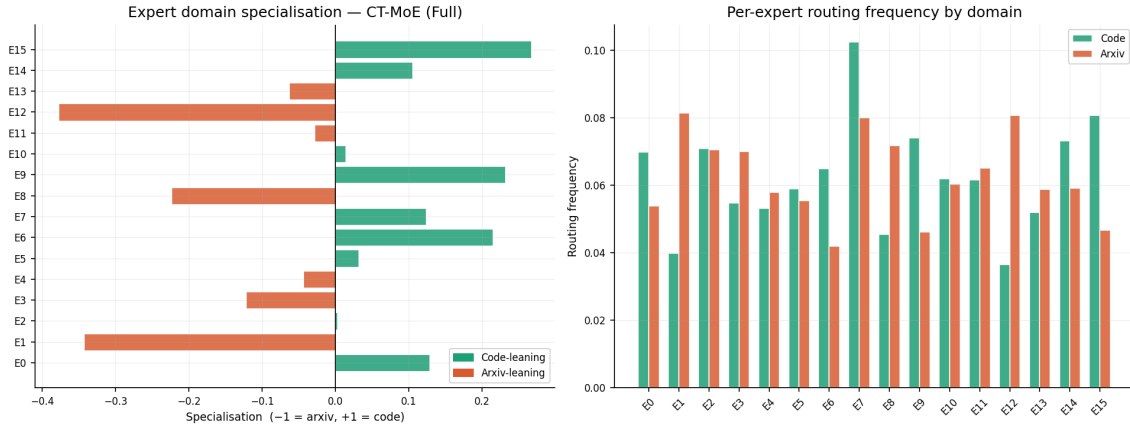


Figure 3: Per-expert domain specialisation for CT-MoE-Full. Left: specialisation score $s_e = (f_e^{\text{code}} - f_e^{\text{arxiv}}) / (f_e^{\text{code}} + f_e^{\text{arxiv}})$. Right: raw routing frequencies by domain. Approximately half the experts develop clear domain preferences without explicit domain supervision.

communication is largely sufficient for the task: any non-trivial weighting over collaboration partners improves on the zero-communication baseline, and the model does not need sharply peaked topology to benefit.

S entropy. The row entropy of $\mathbf{S}$ decreases slowly across training ($2.708 \rightarrow 2.684$; uniform baseline: $\log 15 \approx 2.708$). The topology is diffuse rather than sparse. Strong performance despite near-uniform $\mathbf{S}$ suggests the model benefits from any non-trivial communication structure, not exclusively from sharply peaked collaboration weights. The column structure visible in Figure 2 is not captured by row entropy, since hub experts manifest as column rather than row asymmetries in a row-stochastic matrix.

6.3 Expert Domain Specialisation

Figure 3 shows that experts develop clear domain preferences under CT-MoE-Full. E12, E1, and E8 are the most strongly arxiv-leaning, with E12 showing the largest negative specialisation score ($s_{12} \approx -0.38$). E15, E9, E6, and E7 are the most strongly code-leaning, with E7 routing at roughly 0.10 frequency on code versus 0.08 on arxiv. Approximately half the 16 experts show statistically distinguishable specialisation ($|s_e| > 0.1$), with a roughly 8:6 split between code-leaning and arxiv-leaning experts and 2 near-neutral. This specialisation emerges from the load-balancing objective and the language modelling loss alone—no domain labels or explicit

specialisation supervision are used at any point. The asymmetry in routing frequency for the most specialised experts is consistent with the larger perplexity gain on code: code-specialist experts handle a more focused token distribution, and the message-passing step allows them to share learned representations with generalist experts processing the same sequence.

7 Discussion

Static vs. dynamic topology. A natural extension of CT-MoE is a hypernetwork that generates $\mathbf{S}$ conditioned on a sequence-level representation, enabling input-dependent topologies. Unlike the dynamic approaches in Section 4, a hypernetwork would build $\mathbf{S}$ directly as a low-dimensional output rather than deriving it from pairwise high-dimensional similarities, avoiding geometric saturation.

It is important to distinguish what the static $\mathbf{S}$ does and does not capture. Expert *routing* is fully input-dependent: which experts are selected varies per token, and this selection already produces the domain specialisation observed in Section 6 (E12 and E1 arxiv-leaning; E15, E9, E7 code-leaning). What $\mathbf{S}$ cannot capture is input-dependent *communication structure*: whether the code-specialist experts should collaborate differently among themselves than the arxiv-specialist experts do. Our results show this distinction matters less than might be expected—static $\mathbf{S}$ achieves strong gains regardless—but an input-conditioned $\mathbf{S}$ would in principle allow domain-specific collaboration patterns to emerge, which could be most valuable in settings with sharply different input modalities.

Scale. Our experiments are at 65 M parameters and ~ 50 M training tokens, substantially below production scale. Two competing hypotheses about scaling deserve future investigation. If larger models develop richer expert specialisation [16], the benefit of inter-expert communication may increase with scale. Alternatively, if larger models route more precisely and experts become more independent, the marginal value of communication may decrease. We do not have evidence to distinguish these hypotheses at present.

Limitations. Training uses a specific two-domain mixture; generalisation to homogeneous data or more than two domains is untested. Hyperparameters λ_r , λ_c , τ , and the learning rate multiplier for $\mathbf{S}_{\text{raw}}$ were not tuned by systematic grid search; better values may further improve performance. Our vectorised implementation runs all N experts on every token for implementation simplicity, which does not reflect the sparse computation intended for deployed MoE systems. Compute costs of our implementation are therefore equivalent to a dense model; a truly sparse deployment would require custom CUDA kernels and is left for future work. Finally, with $k=2$ selected experts per token, the message-passing subgraph contains only a single off-diagonal edge per token. Richer communication structures may be accessible at $k \geq 3$.

8 Conclusion

We have presented CT-MoE (Collaborative Topology MoE), a minimal augmentation of sparse MoE layers with a learned static expert collaboration topology. Adding 256 scalar parameters per layer—less than 0.003% overhead on a 65 M model—reduces mixed-domain perplexity by 3.6 points absolute (16.1% relative) over a matched vanilla MoE. A clean four-condition ablation establishes that this gain comes almost entirely from message passing, not routing bias, pointing toward *post-selection communication* as a productive direction for future MoE architecture research. Topology analysis reveals emergent hub experts and low-rank organisation learned without explicit supervision.

The consistent failure of five dynamic topology designs—driven largely by high-dimensional geometric saturation and severed gradient paths—suggests that the research community should carefully examine gradient accessibility when designing learned communication modules for MoE systems. Static learned adjacency matrices are a simple, robust alternative that avoids these failure modes entirely, and may serve as a useful baseline for more expressive future designs.

We release training code, checkpoints, and figures at <https://github.com/pingBenM/CT-MoE>.

References

- [1] M. Artetxe, S. Bhosale, N. Goyal, T. Mihaylov, M. Ott, N. Shazeer, L. Zettlemoyer, et al. Efficient large scale language modeling with mixtures of experts. In *Proceedings of EMNLP*, 2021.
- [2] T. Bai, Y. Yu, L. Huang, Z. Xu, Z. Zhao, and C. Shi. Gmoe: Empowering llms fine-tuning via moe graph collaboration, 2025. URL <https://arxiv.org/abs/2412.16216>.
- [3] A. Clark, D. de Las Casas, A. Guy, A. Mensch, M. Paganini, J. Hoffmann, B. Damoc, B. Hechtman, T. Cai, S. Borgeaud, et al. Unified scaling laws for routed language models. In *International Conference on Machine Learning*, 2022.
- [4] A. Cohan, F. Deroncourt, D. S. Kim, T. Bui, S. Kim, W. Chang, and N. Goharian. A discourse-aware attention model for abstractive summarization of long documents. In *Proceedings of NAACL*, 2018.
- [5] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120): 1–39, 2022.
- [6] D. O. Hebb. *The Organisation of Behaviour*. Wiley, 1949.
- [7] M. Lewis, S. Bhosale, T. Dettmers, N. Goyal, and L. Zettlemoyer. BASE layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning*, 2021.
- [8] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [9] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby. From sparse to soft mixtures of experts. *arXiv preprint arXiv:2308.00951*, 2023.
- [10] D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. C. Humphreys, and A. Santoro. Mixture of depths: Dynamically allocating compute in transformer language models. *arXiv preprint arXiv:2404.02258*, 2024.
- [11] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- [12] L. Tunstall, L. von Werra, and T. Wolf. CodeParrot: A dataset of python code, 2022. URL <https://huggingface.co/datasets/codeparrot/codeparrot-clean>.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [14] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

- [15] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. Dai, Q. Le, and J. Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems*, 2022.
- [16] B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and Q. V. Le. ST-MoE: Designing stable and transferable sparse expert models. In *arXiv preprint arXiv:2202.08906*, 2022.

A Full Training Curves

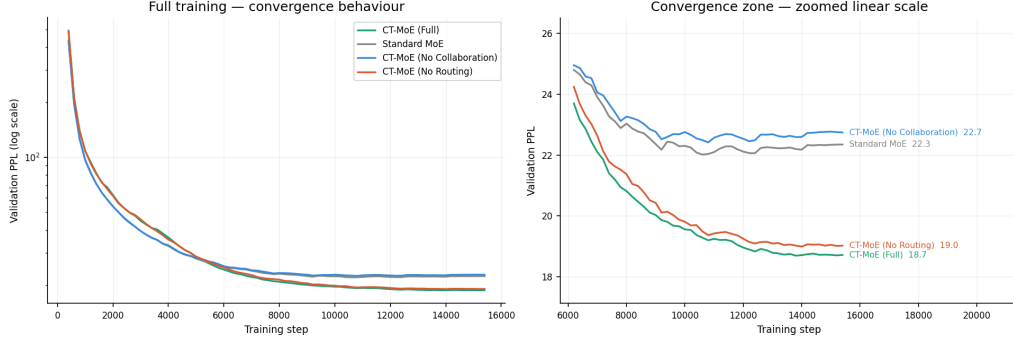


Figure 4: Validation perplexity over training steps, all four variants. CT-MoE-Full and CT-MoE-NoRouting diverge from StandardMoE and CT-MoE-NoCollab from approximately epoch 3 onward, coinciding with the first visible increase in $S_{\max}$ above the uniform baseline.



Figure 5: $\mathbf{S}$ row entropy over training for CT-MoE variants. Starts at the uniform baseline $\log(N-1) \approx 2.708$ and decreases slowly across all variants, confirming learned but diffuse structure. The near-identical entropy trajectories are consistent with the near-identical eigenspectra in Figure 1: all three variants develop similar degrees of row-level structure, with qualitative differences visible in column patterns rather than row entropy.

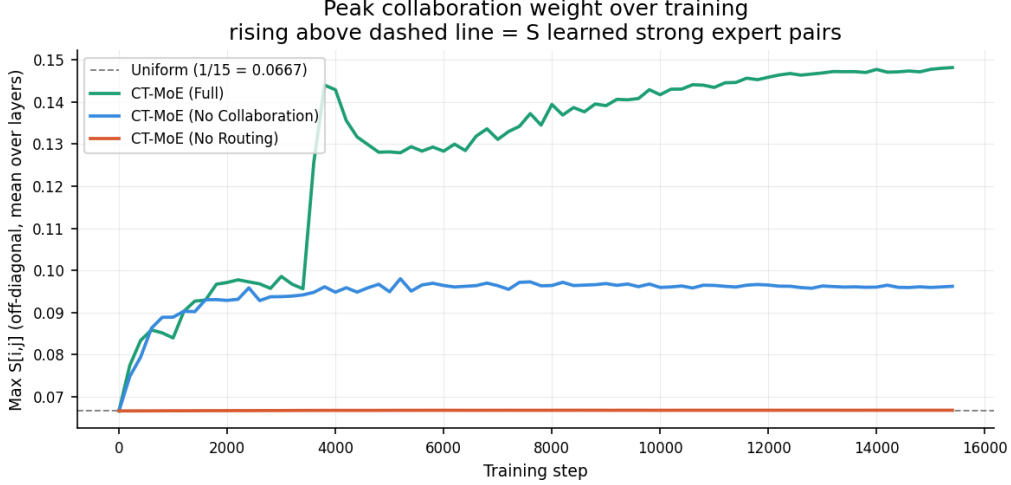


Figure 6: Maximum off-diagonal $\mathbf{S}$ value over training. CT-MoE-Full rises from $1/(N-1) \approx 0.067$ to ≈ 0.148 , with a visible kink at approximately step 3,600 that coincides with an acceleration in perplexity improvement (Figure 4). CT-MoE-NoCollab rises more slowly, consistent with routing bias alone providing insufficient gradient signal to structure $\mathbf{S}$.

B Topology Comparison Across Variants

C Implementation Notes

Vectorised expert execution. All N experts are run in parallel via batched matrix multiplication (`torch.bmm`), and top- k outputs are gathered with `torch.gather`. This avoids Python-level loops over experts and is the primary source of GPU utilisation efficiency in our implementation. As noted in the limitations, this runs all N experts regardless of routing decisions; a production sparse implementation would restrict computation to selected experts only.

Routing bias: column sums, not row sums. Row sums of a row-stochastic $\mathbf{S}$ are identically 1.0 and would add a constant offset to all routing logits, cancelling in the softmax with no effect. The correct routing signal is column sums $\mathbf{S}^\top \mathbf{1}$, which vary as $\mathbf{S}$ learns to assign different incoming weights to different experts.

Separate learning rate for $\mathbf{S}_{\text{raw}}$. Without a learning rate boost, the gradient of the 256-scalar $\mathbf{S}_{\text{raw}}$ is effectively negligible relative to the 65 M-parameter model gradient under AdamW’s adaptive moment estimates. $\mathbf{S}$ shows no measurable deviation from uniform in 10 epochs at equal learning rate. A $100\times$ boost with no weight decay on $\mathbf{S}_{\text{raw}}$ resolves this without affecting the main model’s training dynamics, as the two parameter groups are updated independently.

Symmetrisation and diagonal masking. Both operations are applied at every forward pass rather than once at initialisation, ensuring that $\mathbf{S}$ remains symmetric and diagonal-free throughout training regardless of gradient updates to $\mathbf{S}_{\text{raw}}$. This adds negligible compute ($O(N^2)$ operations against $O(BTND^2)$ for expert execution).

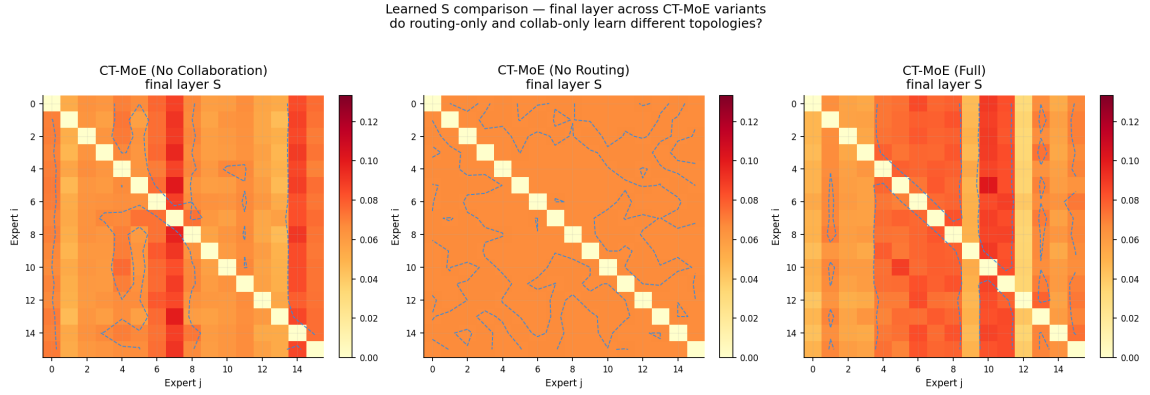


Figure 7: Final-layer $\mathbf{S}$ for CT-MoE-NoCollab, CT-MoE-NoRouting, and CT-MoE-Full side by side. CT-MoE-NoCollab (routing bias only) develops the most pronounced column structure, with strong hubs at columns 0, 7, and 15. CT-MoE-NoRouting (message passing only) is the most diffuse, with the uniform-baseline contour covering most of the matrix—indicating that message-passing gradient alone produces the weakest hub concentration. CT-MoE-Full concentrates on columns 10–11 and 14–15, learning a different hub configuration from either ablation. The divergent hub locations across variants confirm that all three learn non-trivial but distinct collaboration structures, and that which experts become hubs depends on the interplay between routing and communication gradient signals. Blue dashed contour marks the uniform baseline ($1/15 \approx 0.067$).